Ejemplo de aplicación Multiplicación de matrices Optimización

Universidad Nacional de La Plata



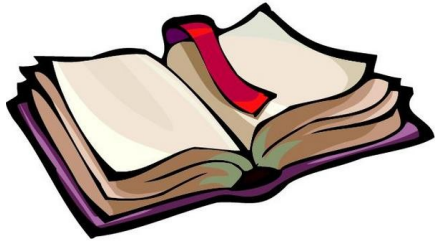
Facultad de Informática

Prof. Dr. Adrian Pousa

Agenda

I. Multiplicación de matrices: Problemas de la solución GPU

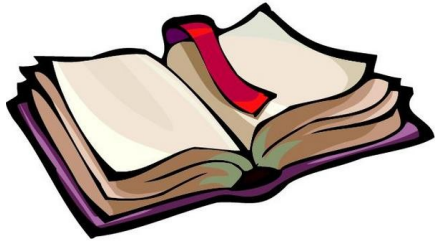
II. Mejoras por el uso de memoria compartida



Agenda

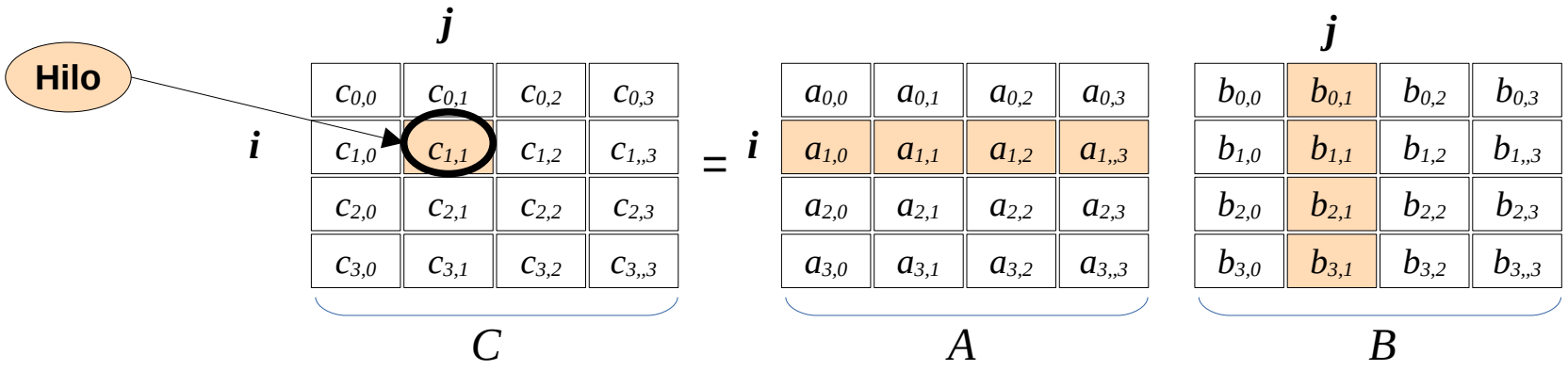
I. Multiplicación de matrices: Problemas de la solución GPU

II. Mejoras por el uso de memoria compartida

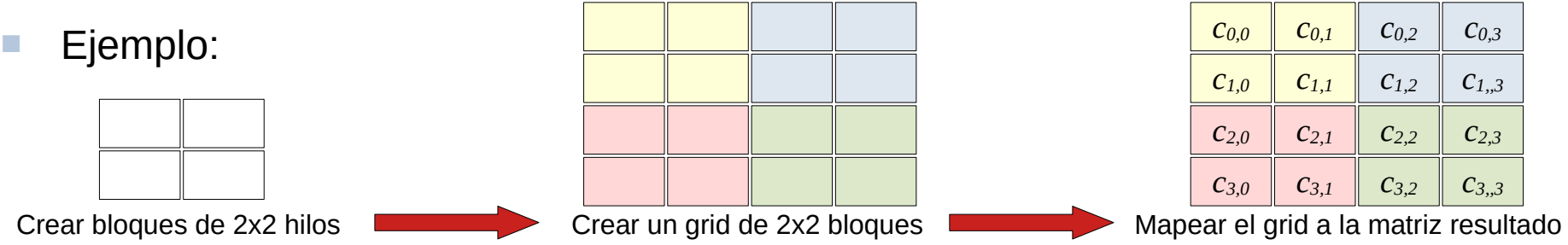


Multiplicación de matrices
Solución en GPU

- La solución en GPU asigna a cada hilo el cálculo de una celda de la matriz resultado.
- Cada hilo CUDA necesita acceder a la fila i de A y la columna j de B.

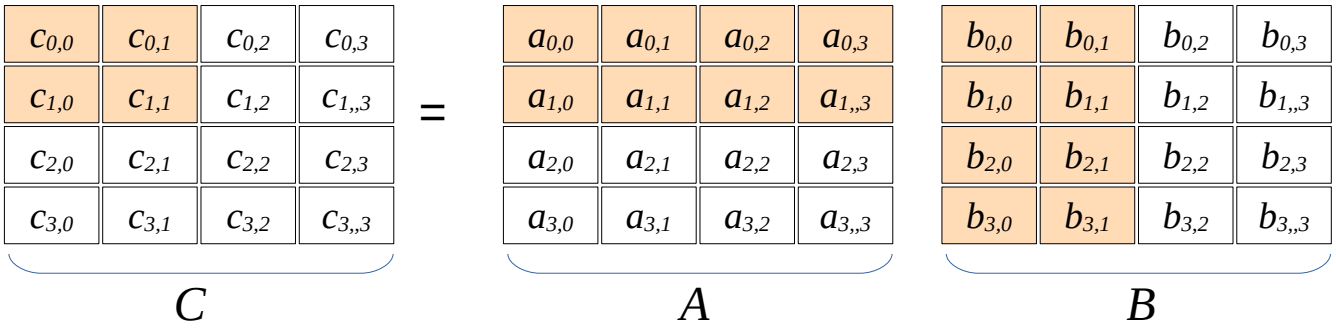


- CUDA permite crear Grids y Bloques bi-dimensionales de hilos que pueden mapearse a la estructura bi-dimensional de las matrices:
  - Ejemplo:



Multiplicación de matrices
Solución GPU - Problemas

- Un bloque de hilos necesitará acceder a ciertas posiciones de las matrices A y B.

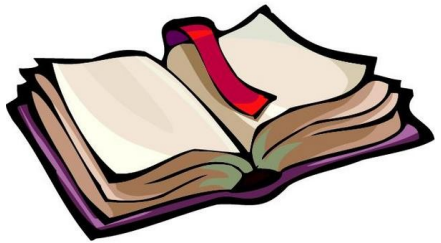


  - Varios hilos de un mismo bloque acceden varias veces a las mismas posiciones en memoria global.
  - Estos accesos a memoria global resultan ser costosos.
  - Podemos evitar la repetición de accesos al mismo dato en memoria global utilizando una memoria más rápida como la memoria compartida.

Agenda

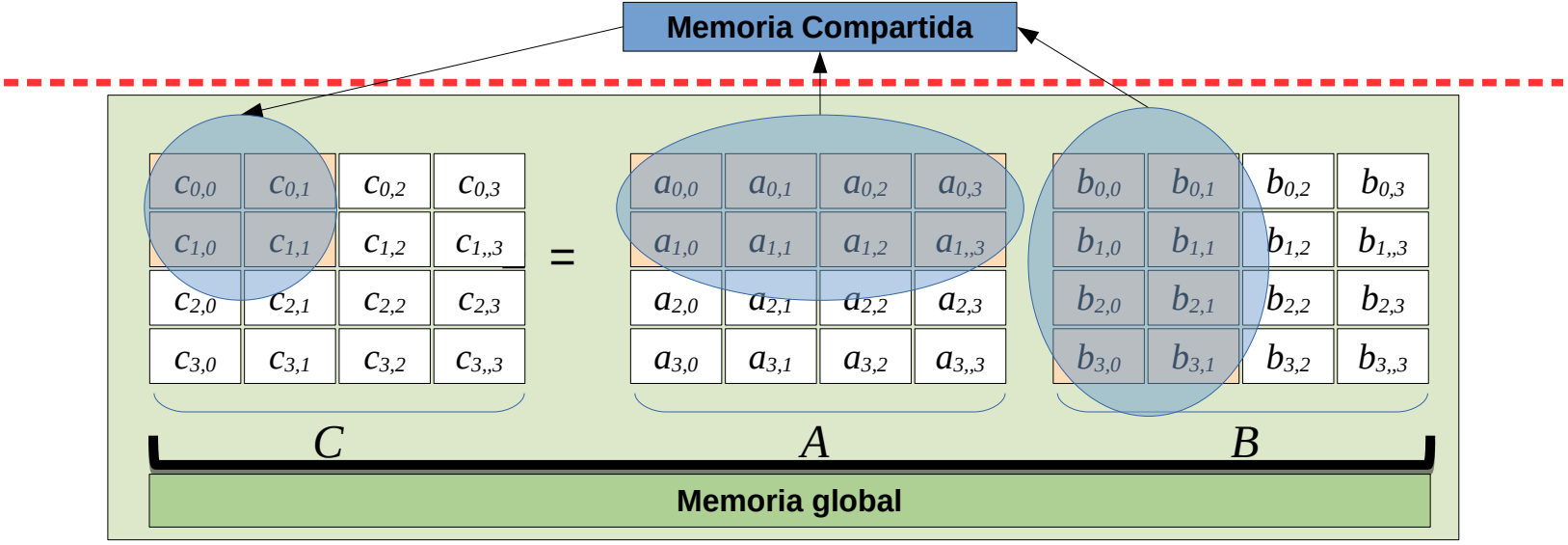
I. Multiplicación de matrices: Problemas de la solución GPU

II. Mejoras por el uso de memoria compartida



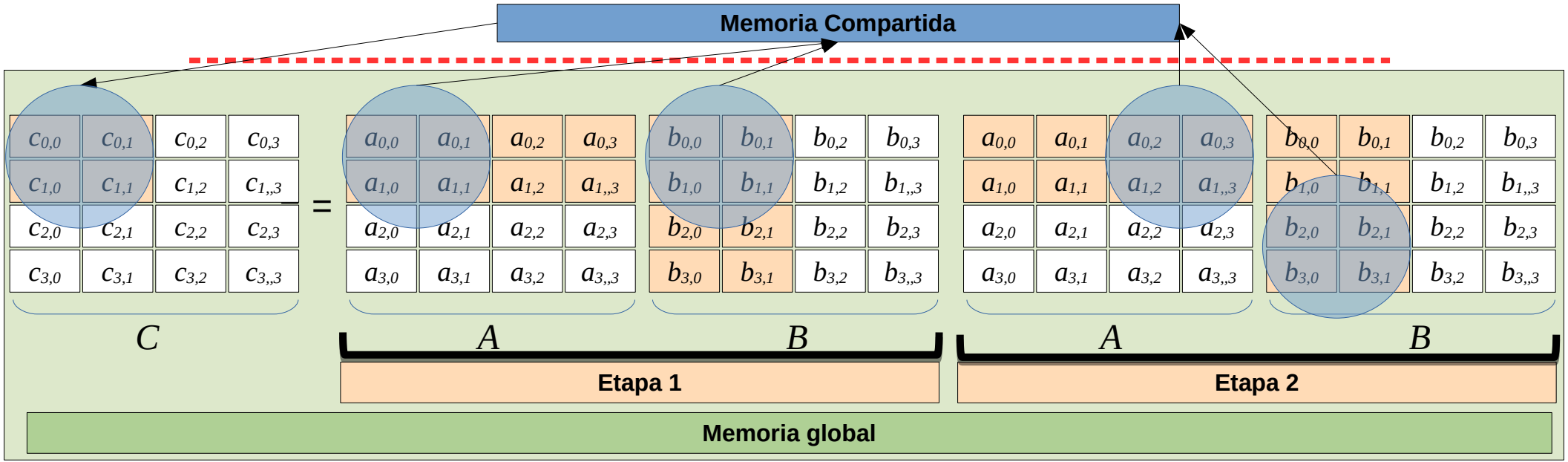
Multiplicación de matrices
Mejoras por el uso de memoria compartida

- La estrategia consiste en:
  - Traer, de forma coalescente, desde memoria global a memoria compartida, los datos necesarios correspondientes a las filas de A y columnas de B.
  - Procesarlos en memoria compartida.
  - El resultado en memoria compartida almacenarlo en la memoria global.



Multiplicación de matrices
Mejoras por el uso de memoria compartida - TILES

- Por las limitaciones de la memoria compartida y para maximizar el rendimiento, las filas de A y las columnas de B se traen y calculan por etapas, de a **TILES**.
- Al completar el cálculo de todas las etapas, se almacena el resultado que reside en memoria compartida en la memoria global.



Multiplicación de matrices
Mejoras por el uso de memoria compartida - Implementación

- Suponemos que cada bloque bidimensional CUDA contiene `BLOCK_SIZE*BLOCK_SIZE` hilos.

```
__global__ void mm_optimizado(float *C, float *A, float *B, int N){
 __shared As[BLOCK_SIZE][BLOCK_SIZE];
 __shared Bs[BLOCK_SIZE][BLOCK_SIZE];

 int fila = blockIdx.y * BLOCK_SIZE + threadIdx.y;
 int columna = blockIdx.x * BLOCK_SIZE + threadIdx.x;
 float c_temp=0.0;
   for(int etapa=0; etapa< N/BLOCK_SIZE; etapa++){
    As[threadIdx.y][threadIdx.x] = A[fila*N + etapa*BLOCK_SIZE + threadIdx.x];
    Bs[threadIdx.y][threadIdx.x] = B[(etapa*BLOCK_SIZE + threadIdx.y)*N + columna];

 __syncthreads();

   for( k=0 ; k<BLOCK_SIZE ; k++ )
    c_temp +=  As[fila,k] * B[k,columna];

 __syncthreads();

   }
       C[fila*N+columna]= c_temp;
}
```

Espacio en memoria compartida para guardar cada TILE

Calcula la fila y columna sobre la que trabajará cada hilo

Lectura coalescente de memoria global a memoria compartida Cada hilo trae un dato por matriz.

Cálculo. En cada etapa deja resultados parciales en c_temp

Escritura coalescente
Cada hilo escribe su resultado almacenado en c_temp a la memoria global

